

REDES NEURONALES MLP - Una introducción.

Verano de 2026

Índice general

1. Definición matemática	4
2. La red más simple	5
2.1. Terminología	5
2.2. Modelado de la red	6
2.3. Evaluación	7
2.3.1. Aprendizaje supervisado	7
2.3.2. Función de coste	7
3. Redes MLP generales	8
3.1. Ecuaciones Generalizadas	8
3.2. Notación Matricial	9
4. Entrenamiento	10
4.1. Pesos y sesgos	10
4.1.1. Inicialización	10
4.2. Caso simple	10
4.3. Descenso Gradiente	12
5. BackPropagation	13
5.1. Objetivo	13
5.1.1. Algoritmos genéticos	13
5.2. Demostración de las ecuaciones de backpropagation	14
5.2.1. Producto Hadamard y error imputado a una neurona	14
5.2.2. Demostración	14
6. Red neuronal en C	16
6.1. Motivación	16
6.2. Diferencias con la programación tradicional	16
6.3. Implementación	17
6.3.1. Dataset	17
6.3.2. Entrenamiento	17
6.3.3. Rendimiento	18

Prólogo

Una aplicación de las matemáticas es modelar fenómenos naturales. En la mayoría de ocasiones estos modelos tratan una simple abstracción de la realidad (una simplificación superficial e incluso burda). Pese a ello, pueden llegar a ser extremadamente útiles. Las leyes de Newton son un ejemplo perfecto. Apoyándose en el cálculo vectorial y las ecuaciones diferenciales permiten predecir, con una extraordinaria exactitud, la evolución de sistemas físicos cotidianos.

Las redes neuronales son nuestro intento de describir algo tan complejo como el cerebro. Puede parecer algo intratable para las matemáticas. Pero debemos recordar que cada una de las acciones que realizamos, consciente o inconscientemente, se deben a millones de impulsos eléctricos. Que, a su vez, resultan de operaciones electroquímicas en nuestras neuronas. Sobre el papel, esta lógica parece replicable. Sin embargo, existen al menos un par de limitaciones que nos impiden almacenar nuestro cerebro en un ordenador.

- Se desconoce parcialmente su funcionamiento.
- Tiene miles de millones de neuronas con sus correspondientes conexiones. Aunque la primera limitación se evaporase seguiríamos necesitando un superordenador.

Por estos motivos simular un cerebro humano está hoy fuera de nuestro alcance. Lo posible es crear una especie de función que presente ciertos paralelismos con el funcionamiento del cerebro humano. Esta función, tras un largo y costoso entrenamiento, producirá soluciones aparentemente inteligentes ante problemas específicos. Espero que este ejemplo tenga algún sentido -no lo tiene ni por asomo-. Tanto un avión de combate como un pequeño ave pueden volar. Ambos logran el mismo objetivo de maneras muy distintas. El avión depende de reactores alimentados por queroseno, mientras que el ave genera sustentación agitando sus alas. Pese a las evidentes diferencias, los dos presentan ciertas similitudes: la necesidad de un cuerpo aerodinámico, o alguna manera de estimar la distancia al suelo para un aterrizaje/posado exitoso.

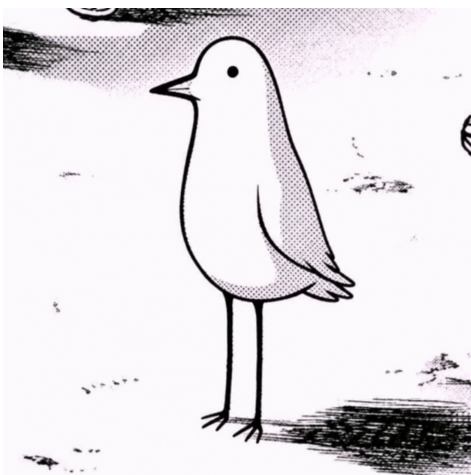


Figura 1: PunPun vs Mig25.

Capítulo 1

Definición matemática

La teoría de grafos es la rama de las matemáticas que estudia los grafos. Las redes de tipo Perceptrón Multicapa (MLP) se pueden representar y definir matemáticamente como estas estructuras. Y, aunque considero que su estudio no es absolutamente necesario en una introducción, cualquier biblioteca moderna de Deep Learning (como PyTorch, TensorFlow o JAX) traduce el código a un grafo antes de ejecutarlo. Por este motivo he tomado la decisión de incluir un diminuto capítulo sobre estos objetos matemáticos.

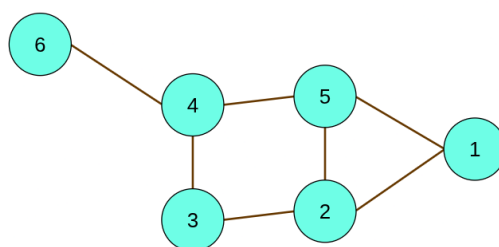


Figura 1.1: Grafo no dirigido.

Los grafos son estructuras formadas por una colección de nodos (cada uno de los círculos) y aristas (cada línea que une los círculos). Una red MLP es un grafo dirigido acíclico (DAG). Vayamos palabra por palabra. Un grafo es una colección de nodos y aristas. Que sea dirigido implica una dirección definida en cada arista, se suelen representar con flechas. Por último, acíclico indica que es imposible regresar al mismo nodo siguiendo el sentido de las aristas.

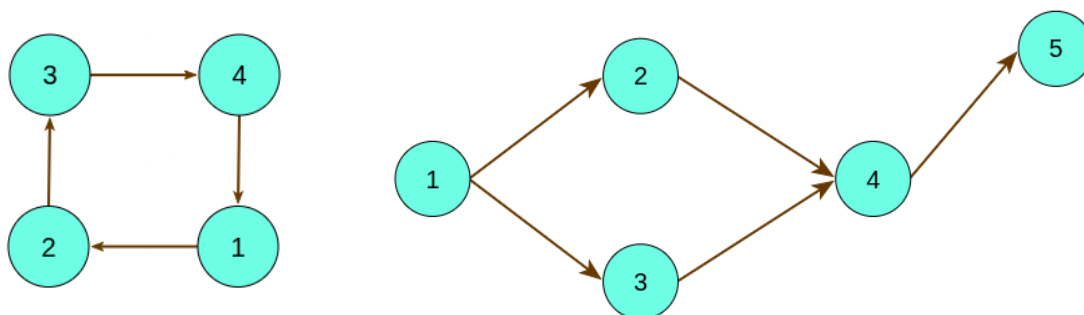


Figura 1.2: Grafo dirigido cíclico y grafo dirigido acíclico.

Capítulo 2

La red más simple

2.1. Terminología

Antes de poder estudiar prácticamente cualquier disciplina debemos familiarizarnos con su terminología. Por ejemplo, a la hora de aprender un deporte necesitamos conocer alguna de las palabras clave sobre las acciones, posiciones o reglas del juego. Las redes neuronales no son una excepción. En esta sección introduciré rápidamente todos los elementos que nos serán necesarios.

- Capa: Columna de nodos en la red. Por convención, la primera capa será la que esté situada más a la izquierda.
- Neuronas de entrada: Donde almacenamos información que consideramos útil para la resolución del problema. Como una codificación de un tablero de ajedrez, los píxeles de una imagen o una altura de vuelo. Son la primera capa de la red.
- Neuronas intermedias: Propagan la información proporcionada por los vectores de entrada hacia la salida.
- Neuronas de salida: Almacenan el resultado de nuestra red. Forman la última capa.
- Sesgo: Escalar asociado a los nodos intermedios y de salida. Se denotan como $\mathbf{b}^x$.
- Peso: Escalar asociado a las aristas. Se denotan como $\mathbf{w}^x$.
- Función de activación: Aplicación no lineal aplicada en la salida de cada neurona.
- Función de coste: Fórmula matemática para determinar qué tan precisas son las predicciones de la red.

2.2. Modelado de la red

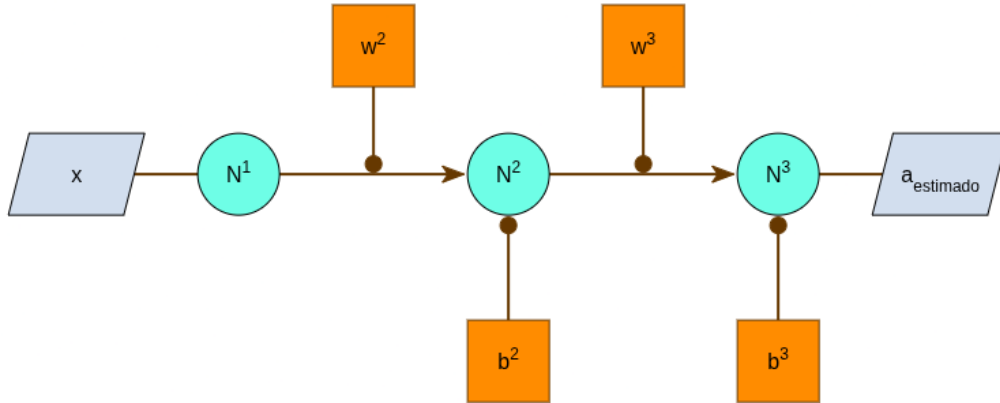


Figura 2.1: Red MLP sencilla.

- Los superíndices de las variables **N w b z a** no indican exponenciación, representan el número de capa al que pertenecen.
- Los bloques con una arista terminada en un círculo no pertenecen a la red, son elementos pedagógicos para indicar donde se encuentran los pesos y nodos

Las redes neuronales son nuestro caballo de batalla para problemas de la vida real. A menudo, tan complicados y complejos que no se pueden resolver con simples directrices de programación: Evaluar posiciones de ajedrez, clasificar caracteres escritos a mano o enseñar a un robot a caminar. Es razonable creer que una red tan sencilla no sería útil ante alguno de estos problemas. Por otra parte, este pequeño calentamiento nos ayudará a comprender mejor escenarios más generales.

Análogamente a lo que ocurre en un cerebro biológico, nuestra red propaga la información desde la entrada (sentidos) a la de salida (señal nerviosa). Esto significa que para calcular el siguiente impulso primero debemos conocer las salidas de todas las neuronas anteriores. Comenzaremos por la segunda capa, ya que la primera solo almacena el valor de entrada **x**. Por este motivo el primer nodo no tiene un sesgo asociado. Podemos pensar que la entrada se propaga sin modificaciones.

Para calcular la señal producida en la segunda neurona haremos una suma ponderada. El segundo nodo toma la señal de activación enviada por la neurona anterior, en este caso **x**, la multiplica por el peso de la arista que une ambas y le suma su sesgo. Esto es: $z^2 = xw^2 + b^2$.

Si propagamos el valor **z** a la siguiente neurona nuestra red se comportará exactamente igual que una regresión lineal, independientemente del número de capas. Ya que una composición de funciones lineales es lineal. Para garantizar que la red aprenda patrones complejos necesitaremos una transformación no lineal. La función de activación. Existen muchas opciones para esta labor: ReLU, tanh o softsign. Nosotros usaremos la sigmoide $\sigma(x)$ por ser la más clásica.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.1)$$

$$\sigma'(x) = e^{-x} \sigma^2(x) \quad (2.2)$$

Esta función fue introducida en un paper de 1986 por Rumelhart e Hinton. Si el valor de entrada en la sigmoide es positivo la salida convergerá rápidamente a 1, en caso contrario a cero. Su objetivo es modelar el estado de una neurona biológica: activa o en reposo. Pese a que la sigmoide fue el estándar durante los años 90, actualmente está obsoleta. Esto es debido a un problema conocido como vanishing gradient. Resumidamente, $\sigma'(x)$ es prácticamente nula para valores altos, un comportamiento que puede ralentizar el aprendizaje.¹

Tras aplicar la sigmoide sobre la suma ponderada de la primera neurona $a^2 = \sigma(z^2)$ finalmente tenemos el valor que será propagado a la siguiente neurona. Siguiendo la misma lógica $a^3 = \sigma(a^2 w^3 +$

¹Para más detalles <http://neuralnetworksanddeeplearning.com/chap3.html>.

$b^3) = \sigma(\sigma(xw^2 + b^2)w^3 + b^3)$. Este último valor será la predicción de nuestra red ante $\mathbf{x}$. En general podemos expresar $\mathbf{z}$ y $\mathbf{a}$ como:

$$z^j = a^{j-1}w^j + b^j \quad (2.3)$$

$$a^j = \sigma(z^j) \quad (2.4)$$

2.3. Evaluación

2.3.1. Aprendizaje supervisado

Para saber qué tan acertada es la predicción de la red definiremos una función de coste $\mathbf{C}$. Esta función comparará la salida de nuestra red con la salida correcta. Podrías pensar -si necesito una forma de conocer la salida correcta, ¿para qué sirve todo esto?-. En realidad, es una pregunta completamente válida.

Para encontrar las salidas correctas se suele recurrir a bases de datos conocidas como datasets. Por ejemplo, alguien podría tomar 70000 imágenes de dígitos escritos a mano y clasificarlos cuidadosamente, indicando que número se corresponde a cada fotografía. Después nosotros usaríamos como valores de entrada los píxeles de la imagen y podríamos compararlos con los previamente etiquetados.

Este tipo de esquema presenta limitaciones importantes. La primera y más evidente es la necesidad de una gran base de datos etiquetada. En la mayoría de ocasiones lo más complicado es construir un dataset que represente fielmente lo que tratamos de modelar. Otro problema importante es que la red será, en el mejor de los casos, tan precisa como nuestro conjunto de datos. En el caso de los dígitos es fácil conseguir un dataset perfectamente etiquetado, pero existen muchos otros problemas donde la solución no es tan sencilla como reconocer un número. Pongo por caso la evaluación de una posición de ajedrez. Si evaluamos posiciones para nuestro dataset con un motor de análisis como Stockfish (uno de los mejores módulos disponibles) nunca evaluará mejor que su entrenador. Y las evaluaciones de Stockfish en general no son correctas. Aunque probablemente sean una excelente aproximación, es indudable que diferirían de la evaluación de Dios sobre la posición. En pocas palabras la calidad de la predicción dependerá en gran medida de nuestro método para conseguir las etiquetas correctas en nuestro dataset.

Existe un último inconveniente algo más sutil que los dos anteriores. Algunos problemas no tienen una forma clara para almacenarse como dataset. He aquí un ejemplo bastante divertido:

<https://youtu.be/0yrH1Sx6pWE>

No existe una manera para determinar que movimiento es óptimo para determinada posición. Si intentamos crear un dataset probablemente sea demasiado ruidoso. La red no aprendería bien. En el mundo del machine learning esta forma de entrenamiento es la que se conoce como aprendizaje supervisado.

2.3.2. Función de coste

Existen diferentes alternativas para nuestra función de coste. Algunas, como Binary Cross Entropy o Categorical Cross Entropy conectan la teoría de la información de Shannon con las redes neuronales. Nosotros nos limitaremos a la función más clásica: Scaled MSE.

$$C(a_{correcto}, a_{estimado}) = \frac{1}{2} \sum_{i=1}^n (a_{estimado}^i - a_{correcto}^i)^2 \quad (2.5)$$

En general esta función recibe dos vectores $a_{correcto}, a_{estimado}$. Pero en nuestra pequeña red cada vector se reduce a un escalar. Así que el sumatorio desaparece y nos queda que $C = \frac{1}{2}(a_{correcto} - a_{estimado})^2$.

Capítulo 3

Redes MLP generales

3.1. Ecuaciones Generalizadas

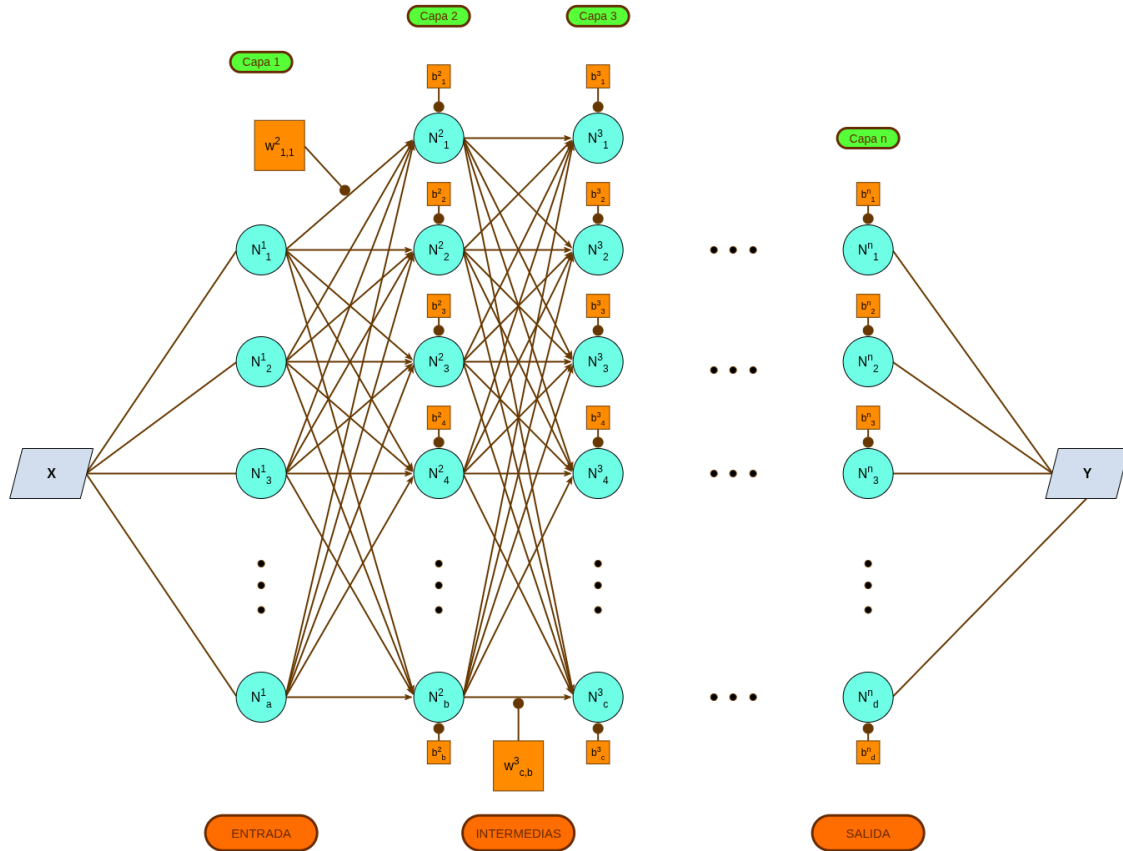


Figura 3.1: Red neuronal MLP general.

- El subíndice indica el número de neurona dentro de la capa. Por convención el primer nodo dentro de cada neurona será el que esté más arriba.
- Introduzco una nueva notación para el peso. El número del primer subíndice indica la neurona de la siguiente capa, mientras que el segundo indica el de la primera. Puede parecer algo extraño pero nos ayudará en el futuro.

La figura 3.1 representa una red MLP en su forma más general, tenemos una cantidad arbitraria de capas y neuronas. Aunque la red crece, la lógica matemática es prácticamente la misma del capítulo

2. La principal diferencia es que ahora tenemos más elementos, por lo que es necesaria una notación con más índices.

Ahora cada neurona tiene un peso asociado con todos los nodos de la siguiente capa, así que en la suma ponderada del capítulo anterior aparecerá un sumatorio que tenga en cuenta todas las señales de la capa previa. Para la neurona i en la capa j suponiendo que la capa anterior tiene n nodos la suma ponderada será:

$$z_i^j = \sum_{k=1}^n w_{i,k}^j a_k^{j-1} + b_i^j \quad (3.1)$$

El único cambio en esta generalización es que ahora tenemos en cuenta todas las señales y pesos anteriores. La activación final producida en la neurona viene dada por:

$$a_i^j = \sigma(z_i^j) \quad (3.2)$$

3.2. Notación Matricial

"No se rían de las notaciones; invéntenlas, son poderosas. De hecho, la matemática es, en gran medida, la invención de mejores notaciones."

Richard Feynman

La notación presentada en el apartado 3.1 es completamente válida aunque resulta bastante pesada. Existe una manera mucho más simple de escribir estas ecuaciones utilizando matrices (que representaré en mayúsculas). Estas nuevas expresiones serán de ayuda para entender de una manera más clara como funcionan las activaciones de cada capa.

Si se fijan en la ecuación 3.1 notarán que la parte del sumatorio es un caso particular de la definición del producto matricial¹. En concreto, es el producto de una matriz y un vector. Por lo que podríamos reorganizar los pesos de cada capa en una matriz $\mathbf{W}$. Las filas de esta matriz contendrán los pesos de cada una de las aristas en cada capa. Por ejemplo, si la primera capa tiene $\mathbf{a}$ nodos y la segunda $\mathbf{b}$. La matriz $\mathbf{W}^1$ tendrá dimensiones $b \times a$. Tendremos tantas filas como neuronas en la segunda capa y cada una de estas filas tendrá necesariamente los $\mathbf{a}$ pesos de las conexiones previas. Además introduciremos también el vector columna $\mathbf{B}$. Cada capa almacenará todos sus sesgos en este vector. Por último, aplicar la función sigmoide sobre una matriz representa la aplicación a todos los elementos de esta. Esto es:

$$\sigma(M) = \begin{bmatrix} \sigma(m_{1,1}) & \sigma(m_{1,2}) & \cdots \\ \sigma(m_{2,1}) & \sigma(m_{2,2}) & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

Después de definir estos convenios estamos listos para reescribir las ecuaciones 3.1 y 3.2 en forma matricial.

$$\boxed{Z^j = W^j A^{j-1} + B^j} \quad (3.3)$$

$$\boxed{A^j = \sigma(W^j A^{j-1} + B^j) = \sigma(Z^j)} \quad (3.4)$$

¹Esta es la razón por la que invertí los índices, si estuviesen en el orden opuesto tendríamos que transponer la matriz $\mathbf{W}$.

Capítulo 4

Entrenamiento

4.1. Pesos y sesgos

Entrenar una red es un problema de optimización. Consiste en minimizar el valor medio producido por la función de coste. Los únicos parámetros de los que depende nuestra red son los pesos y los sesgos. En este capítulo expondré un procedimiento iterativo para modificar estas variables y conseguir un buen rendimiento.

4.1.1. Inicialización

Se suele decir que tanto los pesos como los sesgos se inicializan aleatoriamente. Esto es, en parte, falso. Cuando comienza el entrenamiento se desconocen los valores correctos para cada conexión, pero existen rangos numéricos para asegurar que las señales tengan un comportamiento estable. En el caso de las redes que usan como función de activación la sigmoide o la tangente hiperbólica se sigue la inicialización de Xavier Glorot: ¹

$$W \sim \text{Uniforme} \left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \right) \quad (4.1)$$

$$B = \vec{0} \quad (4.2)$$

Dónde n_{in} es el número de neuronas de la capa anterior y n_{out} el de la siguiente.

4.2. Caso simple

Imagina que quieres encontrar el mínimo absoluto de una función de una sola variable $C(x)$. Como ya sabrás, podemos calcular $C'(x)$ e igualar a cero. Una vez que tenemos la ecuación planteada, podemos tratar de encontrar los puntos que la satisfacen. En la siguiente función:

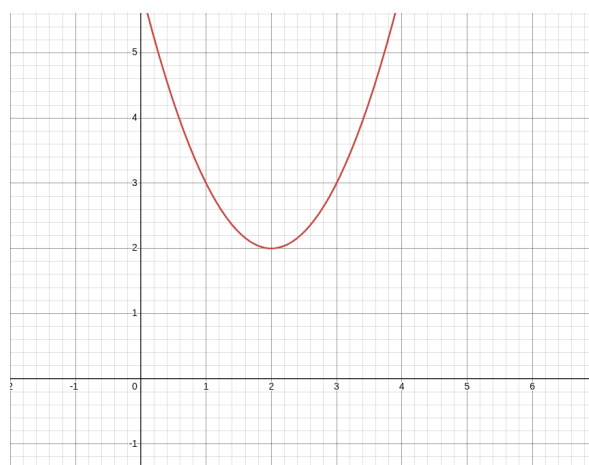


Figura 4.1: $C(x) = x^2 - 4x + 6$.

¹<https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf>

La derivada de nuestra función de coste es $C'(x) = 2x - 4$ ahora la igualamos a cero para $2x_{min} - 4 = 0 \Rightarrow x_{min} = 2$. Sin embargo, calcular el mínimo de manera analítica se vuelve impracticable debido a la enorme complejidad de las redes. Es necesario un método alternativo.

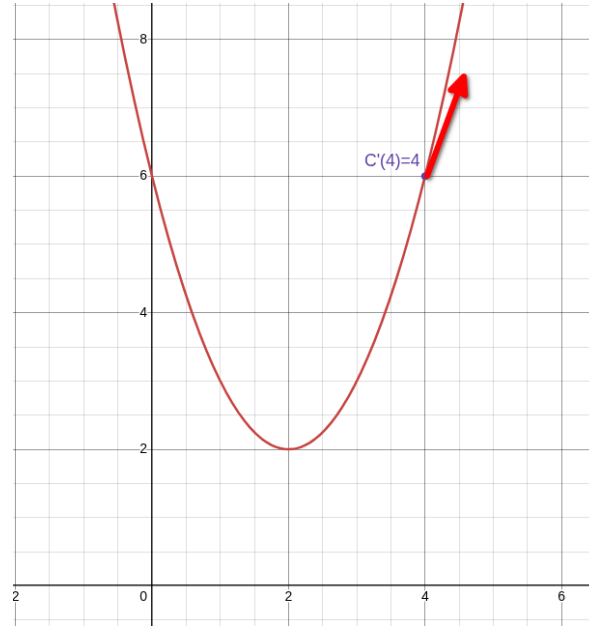


Figura 4.2: Presentación del método iterativo en monovariable.

En este nuevo método calculamos $C'(x)$ y elegimos un punto arbitrario, en este caso $x=4$. Ahora en lugar de resolver la ecuación evaluamos $C'(4)$. Para buscar un mínimo, si la pendiente es positiva debemos movernos hacia la izquierda, y viceversa. Podemos imaginarlo como una bola que cae por una colina. ² Este procedimiento no garantiza llegar al mínimo absoluto de la función, ya que podríamos acabar antes en uno relativo. Todo depende de las condiciones iniciales.

$$x_{n+1} = x_n - \eta C'(x_n) \quad (4.3)$$

En esta expresión, η es tan solo una constante entre 0 y 1 que nos permite *regular el paso* que tomamos, podría ocurrir que el valor de la derivada cerca del mínimo sea elevada. Lo que causaría que la siguiente iteración comenzase lejos del mínimo. Esta clase de comportamiento causará que nunca acabe de converger en el mínimo. En este escenario si definimos un valor de η lo suficientemente pequeño evitaríamos este comportamiento, ya que los pasos de nuestro algoritmo serían mucho más precisos. Sin embargo, un η pequeño corre el riesgo de caer en un mínimo local alto. En la práctica al comienzo del entrenamiento se eligen valores para η altos con el objetivo de bordear los mínimos relativos. Cuando la solución es lo suficientemente buena se establece una η baja para asegurar la convergencia.

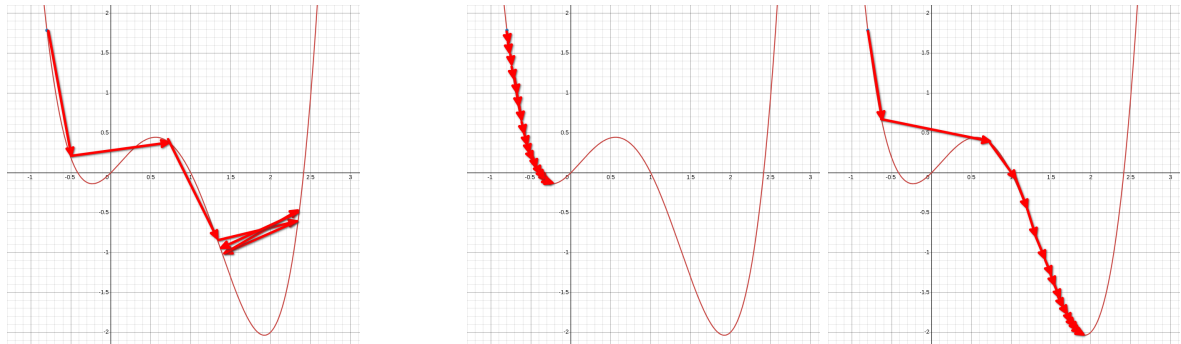


Figura 4.3: η grande/pequeña/adaptada.

²Este ejemplo no es una coincidencia ya que la fuerza conservativa es el negativo de la derivada de la energía potencial con respecto a la posición $F(y) = -\frac{dU}{dy}$.

4.3. Descenso Gradiente

El gradiente es la generalización multivariable de la derivada. Si $f(\vec{x})$ es un campo escalar entonces el gradiente de f se define como:

$$\nabla f(\vec{x}) = \left(\frac{\partial f(\vec{x})}{\partial x_1}, \dots, \frac{\partial f(\vec{x})}{\partial x_n} \right) \quad (4.4)$$

Para cada punto el gradiente de f devuelve un vector. Este vector indica la dirección en que la función escalar aumenta con mayor rapidez y su magnitud representa esa tasa máxima de cambio. Esta propiedad nos permite generalizar el segundo método de la sección 4.2.

$$\boxed{W_{n+1}^{\vec{}} = W_n^{\vec{}} - \eta \nabla C(W_n^{\vec{}})} \quad (4.5)$$

- $-\nabla C(\vec{W})$. Naturalmente, el sentido contrario al de mayor crecimiento es el de mayor decrecimiento.

Al igual que antes este algoritmo no garantiza encontrar el mínimo absoluto. En la práctica es imposible debido al gigantesco espacio dimensional de las redes neuronales habituales, existen demasiadas variables. Pero repetir este algoritmo las suficientes veces garantiza encontrar un mínimo local.

Nosotros usaremos la técnica de descenso del gradiente en nuestra función de coste. Scaled MSE es una función escalar que depende de los pesos y sesgos. Encontrar los pesos y sesgos que hacen el error mínimo causará que nuestra red dé predicciones acertadas, en otras palabras, aprenderá a resolver el problema.

Capítulo 5

BackPropagation

5.1. Objetivo

Parece que ya estamos listos para entrenar a nuestra red neuronal. Podemos inicializar nuestros pesos siguiendo el criterio de Xavier Glorot, y seguidamente usar descenso del gradiente para optimizarlos. Pero todavía existe un último problema: el cálculo numérico del gradiente. Una primera idea sería aplicar la definición de derivada para cada variable de nuestra red.

$$\frac{\partial C}{\partial w} = \lim_{x \rightarrow 0} \frac{C(w+x) - C(w)}{x} \approx \frac{C(w+\epsilon) - C(w)}{\epsilon} \quad (5.1)$$

Con este enfoque debemos realizar esta operación para cada uno de los pesos y sesgos de nuestra red. Pero en una red con millones de pesos este procedimiento es extremadamente lento. Por suerte existe un algoritmo para calcular el gradiente con un coste computacional bajo. BackPropagation.

BackPropagation es el estándar de la industria, se utiliza en el 99% de los entrenamientos. Además, existe toda una familia de algoritmos de optimización que lo usan: Adam, SGD, Momentum, GD(descenso del gradiente). Por lo que la correcta comprensión de este algoritmo es fundamental para seguir avanzando en el estudio de redes neuronales.

5.1.1. Algoritmos genéticos

Aunque los optimizadores por gradiente son la opción mayoritaria, no son el único camino. Debo hacer una pequeña mención a los algoritmos evolutivos. Existen algoritmos inspirados en la evolución biológica para ajustar los pesos. Aunque estos algoritmos son menos eficientes en la mayoría de situaciones, personalmente siempre me han parecido algo entrañable. Además tienen sus aplicaciones en sistemas de control, robótica, o, incluso, videojuegos. Donde BackPropagation es más difícil de aplicar.

La NASA ha usado algoritmos genéticos en el diseño de antenas que requieren un patrón de radiación extremadamente específico. Estos diseños han demostrado ser superiores a los conseguidos por los métodos tradicionales de diseño. Y, aunque siempre he tenido una atracción muy fuerte hacia lo simple, me fascina la capacidad de estos algoritmos para sacar a la luz soluciones desde la más profunda oscuridad de la naturaleza.

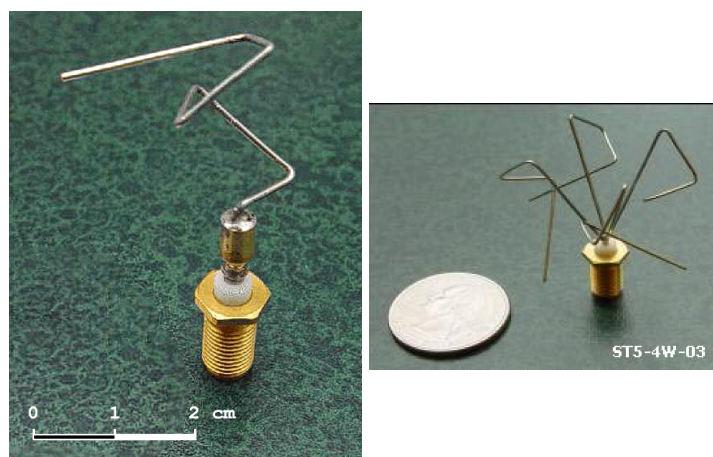


Figura 5.1: Antenas creadas mediante algoritmos evolutivos.

5.2. Demostración de las ecuaciones de backpropagation

5.2.1. Producto Hadamard y error imputado a una neurona

Antes de comenzar con la demostración presentaré un par de conceptos que nos serán necesarios. El primero de ellos es el producto Hadamard. En esta operación se multiplica cada elemento de la primera con el que está en la misma posición de la segunda, es necesario que ambas matrices tengan las mismas dimensiones.

$$A \odot B = \begin{pmatrix} a_{11}b_{11} & a_{12}b_{12} & \cdots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \cdots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \cdots & a_{mn}b_{mn} \end{pmatrix}. \quad (5.2)$$

Además definiremos el error imputado a cada neurona como:

$$\delta_j^l \triangleq \frac{\partial C}{\partial z_j^l} \quad (5.3)$$

Donde l indica la capa y j la neurona dentro de ella. Esta expresión modela como varía el error total de la red si modificamos la suma ponderada de una sola neurona. Aparecerá numerosas veces en las demostraciones.

5.2.2. Demostración

Primera ecuación.

$$\delta^L = \nabla_a C \odot \sigma'(Z^L) \quad (5.4)$$

Por definición $\delta^L = \frac{\partial C}{\partial z^L}$. Además, $C = f(\sigma(Z^L))$. Por la regla de la cadena $\frac{\partial C}{\partial z^L} = f'(A^L)\sigma'(Z^L) = \nabla_a C \odot \sigma'(Z^L)$. Esta ecuación nos permite calcular el error imputado a las neuronas de la última capa L .

Segunda ecuación.

$$\delta^l = (W^{l+1})^T \delta^{l+1} \odot \sigma'(Z^l) \quad (5.5)$$

Ahora calcularemos como afecta el error perteneciente a una neurona de una capa arbitraria a la salida. $\frac{\partial C}{\partial z_j^l} = \sum_k \frac{\partial C}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_j^l}$. El sumatorio aparece porque la neurona z_j^l está conectada con todas las de la capa $l+1$, por lo que una variación en ella les afecta. El primer término $\frac{\partial C}{\partial z_k^{l+1}} = \delta_k^{l+1}$ y z_k^{l+1} por la ecuación 3.1 es $z_k^{l+1} = \sum_j w_{k,j}^{l+1} a_j^l + b_k^{l+1} = \sum_j w_{k,j}^{l+1} \sigma(z_j^l) + b_k^{l+1}$. Por lo que el segundo miembro: $\frac{\partial z_k^{l+1}}{\partial z_j^l} = w_{k,j}^{l+1} \sigma'(z_j^l) \Rightarrow \frac{\partial C}{\partial z_j^l} = \sum_k \delta_k^{l+1} w_{k,j}^{l+1} \sigma'(z_j^l) \Rightarrow \delta^l = (W^{l+1})^T \delta^{l+1} \odot \sigma'(Z^l)$ Esta ecuación combinada con la primera nos permite calcular el error imputado a cada neurona de cualquier capa.

Tercera ecuación.

$$\frac{\partial C}{\partial B^l} = \delta^l \quad (5.6)$$

$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial b_j^l}$. Donde $\frac{\partial C}{\partial z_j^l} = \delta_j^l$ y $z_j^l = \sum_k w_{j,k}^l a_k^{l-1} + b_j^l$ y $\frac{\partial z_j^l}{\partial b_j^l} = 1 \Rightarrow \frac{\partial C}{\partial b_j^l} = \delta_j^l \Rightarrow \frac{\partial C}{\partial B^l} = \delta^l$. Esta ecuación nos permite calcular los gradientes de los sesgos de cada capa.

Cuarta ecuación.

$$\frac{\partial C}{\partial W^l} = \delta^l (A^{l-1})^T \quad (5.7)$$

$\frac{\partial C}{\partial w_{j,k}^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial w_{j,k}^l}$. Al igual que antes $\frac{\partial C}{\partial z_j^l} = \delta_j^l$. Lo que cambia es que $\frac{\partial z_j^l}{\partial w_{j,k}^l} = a_k^{l-1} \Rightarrow \frac{\partial C}{\partial w_{j,k}^l} = \delta_j^l a_k^{l-1} \Rightarrow \frac{\partial C}{\partial W^l} = \delta^l (A^{l-1})^T$. Esta última ecuación nos permite calcular el gradiente de los pesos en cada capa.

Ecuaciones BackPropagation.

$$\begin{aligned} \delta^L &= \nabla_a C \odot \sigma'(Z^L) \\ \delta^l &= (W^{l+1})^T \delta^{l+1} \odot \sigma'(Z^l) \\ \frac{\partial C}{\partial B^l} &= \delta^l \\ \frac{\partial C}{\partial W^l} &= \delta^l (A^{l-1})^T \end{aligned} \quad (5.8)$$

Es fácil ver que estas cuatro ecuaciones nos permiten calcular *hacia atrás* los gradientes para todos los pesos y sesgos de la red.

Capítulo 6

Red neuronal en C

6.1. Motivación

Cuando estudio algo nuevo siempre intento crear un proyecto relacionado. En este caso creé una [biblioteca sencilla en C](#) para crear, entrenar y exportar redes neuronales MLP. Aunque es mucho menos potente cualquier API moderna, es ideal para alguien que quiere aprender como funcionan las redes desde los fundamentos. En el propio código se puede leer el álgebra lineal que está detrás de cada paso. Mientras que con cualquier biblioteca moderna estos detalles son completamente transparentes.

Esta falta de abstracción no añade complejidad al código. Este pequeño proyecto está especialmente diseñado para que cualquier persona con conocimientos básicos en programación pueda experimentar sin problemas. Si ya tienes experiencia en programación puedes crear tu propia implementación. Es un gran ejercicio para interiorizar todo lo aprendido.

6.2. Diferencias con la programación tradicional

Las redes neuronales presentan distinción fundamental con la programación 'tradicional': Cuando resolvemos un problema con un ordenador necesitamos definir un algoritmo. Una serie de pasos a seguir que solucionarán nuestro problema. Pensemos, por ejemplo, en cualquier algoritmo de ordenación. Uno muy sencillo podría ser:

```
//Algoritmo (mediocre) para ordenar de manera ascendente un array

Repetir hasta que la condición 1 sea falsa {

    1. Iterar sobre mi array y comprobar si no hay ningún número
    mayor situado antes que uno menor.
    2. Si la condición 1 se cumple intercambiar el orden de los dos
    números encontrados.

}
```

Curiosamente, cuando solucionamos un problema con una red neuronal "no sabemos cómo lo hace". Conocemos perfectamente como calcular matemáticamente la salida de cualquier red. Pero desconocemos el 'algoritmo' que sigue para resolver el problema. Esto es útil en situaciones en las que el algoritmo a seguir no es claro. Una aplicación clásica es OCR(detección de caracteres en una imagen). Imagina que debemos escribir un algoritmo que sea capaz de clasificar los números en una imagen. Un primer pensamiento ingenuo sería analizar la densidad de píxeles de la fuente. En el caso del 1 encontraremos la mayoría de los píxeles en una franja vertical, un 0; en cambio, tendrá una distribución más amplia. Sin embargo, esta idea presenta un problema fundamental: ¿Qué pasa si cambian la tipografía? O lo que es peor, ¿qué pasa si ni siquiera existe una tipografía(por estar escritos a mano)?. Las redes neuronales nos darán una solución mucho más robusta. Incluso para tipografías o letras con las que no haya sido entrenada.

6.3. Implementación

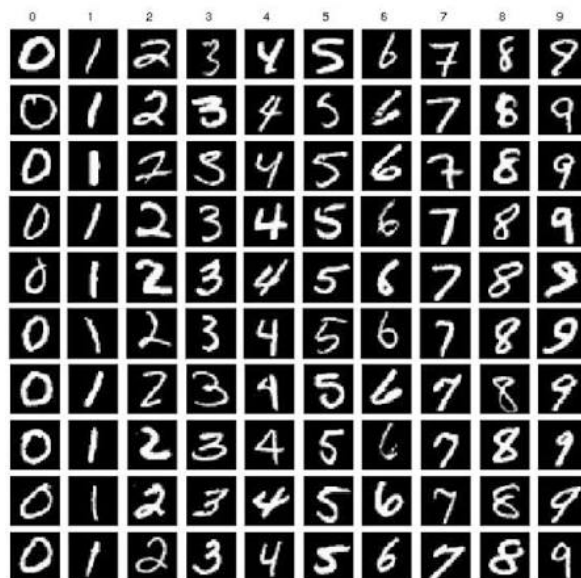


Figura 6.1: Dataset MNIST

En este último apartado del manual llevaremos a la práctica lo aprendido. Haremos el *hola mundo* de las redes neuronales, clasificar dígitos escritos a mano. Usaremos el dataset MNIST, una base de datos clásica con 70.000 imágenes en escala de grises. La primera capa recibe los 784 píxeles, así que necesita esa misma cantidad de neuronas. La de salida tendrá 10 neuronas para cada uno de los números. He aquí el código y [archivos](#).

6.3.1. Dataset

En este caso partimos de un dataset ya creador; esto simplifica mucho nuestra labor. Pero si quieres usar desCent debes de conocer algunos detalles necesarios en cualquier implementación.

[illegible]

Estas son las tres primeras líneas de el dataset *mnist.train.csv*. La primera línea indica el número de filas y de columnas. Tenemos un total de 60000 muestras y cada una de ellas tiene 785 elementos. El primero de estos elemetos es la etiqueta, en este caso es el número de la imagen, los siguientes 784 dígitos son los píxeles de la fotografía.

6.3.2. Entrenamiento

trainNetwork.c

```
//Comando de compilación: gcc trainNetwork.c ml.c algebra.c -o trainNetwork -lm -O3 -march=native -flto
#include <stdio.h>
#include "ml.h"
#include "algebra.h"

int main() {
    int layers[] = {784, 16, 16, 10}; //Número de neuronas por cada capa
    short nLayers = sizeof(layers) / sizeof(layers[0]);

    char csvPath[] = "/home/passpow/Blog/MLP/mnist_train.csv";
    char outPath[] = "/home/passpow/Blog/MLP/network/";

    int rows, cols;
    float* dataset;
```

```

Layer* network = createNetwork(nLayers, layers);
printNetwork(network);

if (importMatrixCsv(&rows, &cols, &dataset, csvPath) != 0) {
    printf("Error reading dataset.\n");
    return -1;
}
printf("Dataset cargado correctamente!\n");
printf("Número de filas: %d\n", rows);
printf("Número de columnas: %d\n", cols);

// Comenzamos con una eta grande que vamos reduciendo
for (int i = 10; i <= 10000; i *= 10) {
    trainNetwork(
        network,
        dataset,
        1.0f / i,    // eta
        1,          // Cada muestra solo contiene una etiqueta, el número
        784,         // Cada muestra tiene 784 píxeles de entrada
        0,          // los valores de entrada están entre 0 y 255 por ser imágenes
        255,
        rows        // número de muestras
    );
}

exportNetwork(network, outPath);

printf("Red exportada en %s\n", outPath);

return 0;
}

```

Este es el código que he usado para entrenar la red neuronal. Comienza definiendo el número de neuronas por cada capa.

```
int layers[] = {784, 16, 16, 10};
```

Nuestra red recibirá 784 píxeles, los propagará por dos capas intermedias de 16 neuronas y finalmente activará 10 neuronas de salida para poder distinguir entre los 10 dígitos. Interpretaremos que la neurona con un valor de salida más alto será la seleccionada por la red.

6.3.3. Rendimiento

testNetwork.c

```

//Comando de compilación: gcc testNetwork.c ml.c algebra.c -o testNetwork -lm -O3 -march=native -flto
#include <stdio.h>
#include <stdlib.h>
#include "ml.h"
#define testValues 1000
int main() {
    int layers[] = {784, 16, 16, 10};
    short nLayers = sizeof(layers) / sizeof(layers[0]);

    char csvPath[] = "/home/passpow/Blog/MLP/mnist_test.csv";
    char outPath[] = "/home/passpow/Blog/MLP/network/";

    int rows, cols;
    float* dataset;
    float* outputVector=malloc(layers[nLayers-1]*sizeof(float));

    Layer *network = createNetwork(nLayers, layers);

```

```

if (importMatrixCsv(&rows, &cols, &dataset, csvPath) != 0) {
    printf("Error reading dataset.\n");
    return -1;
}
importNetwork(network, outPath);
int success=0;
for (int row=0; row<testValues; row++){
    int actualNumber = dataset[row*785+1];
    getOutput(network, &dataset[row*785+1], &outputVector, 0.f, 255.f);
    int estimatedNumber = vectorMax(outputVector, layers[nLayers-1]);
    printf("EXPECTED/OUTPUT: %d/%d \n", actualNumber, estimatedNumber);
    printf("OUTPUT OF THE NETWORK:\n");
    for (int j=0; j<layers[nLayers-1]; ++j){
        printf("%f\n", outputVector[j]);
    }
    if (estimatedNumber==actualNumber) success++;
}
printf("La red ha acertado %d de %d\n", success, testValues);
free(dataset);
free(outputVector);

return 0;
}

```

Este programa muestra por pantalla el valor esperado, el estimado por la red y el vector de activación A^L . Al finalizar indica los aciertos.

```

EXPECTED/OUTPUT: 9/9
OUTPUT OF THE NETWORK:
0.005907
0.000134
0.003364
0.009528
0.023872
0.000100
0.000005
0.440249
0.000101
0.665431
La red ha acertado 936 de 1000

```

Figura 6.2: vector de salida para la imagen número 1000 de *mnist_test.csv*

Una red formada por tan solo dos capas de dieciséis neuronas cada una y 40 líneas de código consigue una tasa de acierto del 93.6%, mientras que una API moderna que posee optimizadores avanzados con una red mucho mayor ronda el 97%-98%.